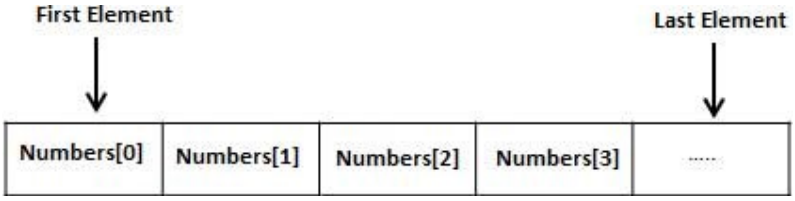
UNIT-III
ARRAYS & STRINGS

ARRAYS

An array is defined as the collection of similar type of data items stored at contiguous memory locations. Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc. In an array, a data element can be randomly accessed by using its index number.

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.



Properties of Arrays - The array contains the following properties.

- Each element of an array is of the same data type and carries the same size, i.e., int = 4 bytes.
- Elements of the array are stored at contiguous memory locations where the first element is stored at the first and base addressed memory location.
- Elements of the array can be randomly accessed by calculating the address of each element of the array with the given base address and the size of the data element.

Advantages of Arrays - The array contains the following advantages.

- Code Optimization: Less code to access the data.
- Ease of traversing: By using the for loop, one can retrieve the elements of an array easily.
- Ease of sorting: To sort the elements of the array, a few lines of code only needed.
- Random Access: Can access any element randomly using the array.

DECLARING ARRAYS

To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array as follows −

type arrayName [arraySize];

This is called a **SINGLE-DIMENSIONAL ARRAY**. The **arraySize** must be an integer constant greater than zero and **type** can be any valid C data type and **arrayName** must be a valid Identifier.

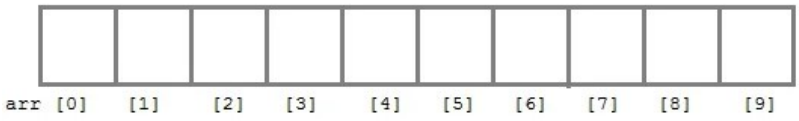
/* Example of array declaration */

```
char a[5];    /* char type value array */
float ar[9];  /* float type value array */
int arr[10];  /* int type value array */
```

In the code above, in the first line, char is the data type, a is the name of array and 5 is the size.

In the next line, float is the data type, ar is the name of the array and size of array is 9.

And in the last line of code, int is the data types, and arr is the name of the array and 10 is the size of array.



It means array arr can only contain 10 elements of int type.

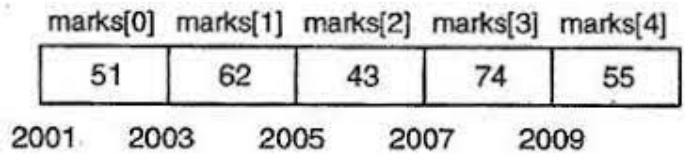
The index of an array starts from 0 to arraySize-1 i.e first element of any array will be stored at arr[0] address and the last element will be at arr[arraySize-1].

Array Memory Model

if an array is declared as,

int marks[5];

then the **memory model or memory representation** of such array is shown below.



Memory Representation of `marks`

INITIALIZING ARRAYS

After an array is declared it must be initialized. Otherwise, it will contain garbage values(any random value). An array can be initialized at either compile time or at runtime.

Compile-time Array initialization

Compile time initialization of array means providing the value for the array in the code when array is declared. In this method of array declaration, the compiler will allocate an array of size equal to the number of the array elements or values given.

data_type array_name [size] = {value1, value2, ... valueN};

Here the **Size** is optional, because the complier will allocate an array of size equal to the number of the array elements or values given.

/* Here are a few examples */

int marks[4] = { 67, 87, 56 }; // **integer array initialization with four elements with default initialization of marks[3]=0**

float area[] = { 23.4, 6.8, 5.5 }; // float array initialization with 3 elements

int marks[4] = { 67, 87, 56, 77, 59 }; // Compile time error

if more values(array elements) are given than the declared array size than the compiler will give an error.

Runtime Array initialization

An array can also be initialized at runtime using scanf() function. This approach is usually used for initializing large arrays, or to initialize arrays with user specified values.

For example, to insert element at specific index,

scanf("%d", &arr[3]); // will insert element at index 3, i.e. 4th position

Another example to insert elements at each index,

```
for(int i = 0; i < 10; i++)
    Arr[i] = i*2;
```

Here **i** refers to the index of the elements, starting from 0 and ending at 9 which is less than 10. In each element, the value of its index*2 is stored.

Another Example to read elements at each index according to user, the following code works.

```
for(int i = 0; i < 10; i++)
    scanf("%d", &Arr[i]);
```

ARRAY INDEXING

Accessing Array Elements - Array indexing is used to **access the array elements** using the index of the element. Since an array is stored contiguously in the memory, it has indices starting from ”0” to “array_size – 1. These array indices are used to access any element of the array in the following way:

array_name[index]

The **index** of the element to be accessed is specified within square brackets “**[]**”. The range of the index is- integers in the range [0 to ArraySize-1].

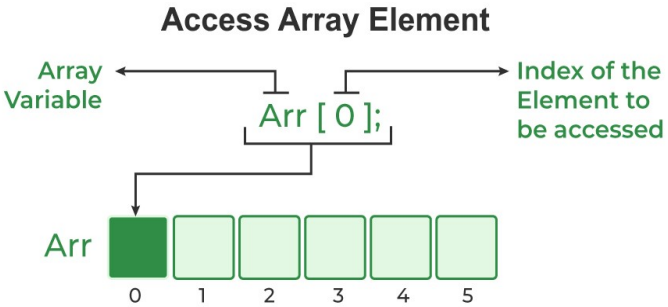
Examples:

int my_array[6];

my_array[0] = 100; // access 1st element

my_array[2] = 300; // access 4th element

my_array[5] = 600; // access last element



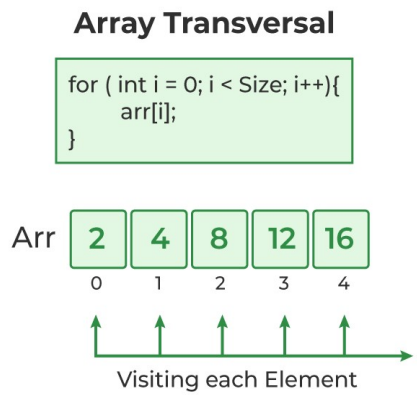
Update Array Element – Array indexing can also be used to update the value of an element at the given index **i** as shown in general form below.

array_name[i] = new_value;

Array Traversal – Array indexing can also be used to Array Traversal. Traversal is the process in every element of the array is accessed. Generally a for loop is used to iterate through each element of the array.

Example Array Traversal using for Loop

```
for (int i = 0; i < N; i++)
{
   array_name[i];
}
```



The following program demonstrates array indexing.

```
void main()
{
   // array declaration and initialization
   int arr[] = { 10, 20, 30, 40, 50 };
    // modifying element at index 2
   arr[2] = 100;
    // traversing array using for loop
   printf("Elements in Array: ");
   for (int i = 0; i < 5; i++)
      printf("%d ", arr[i]);
}
```

Output:

Elements in Array: 10 20 100 40 50

PROGRAMS WITH ARRAY OF INTEGERS

Calculate the Sum of Array Elements

```
void main()
{
   int arr[5],sum=0,i;
   printf("Enter 5 array elements:");
   for(i = 0; i < 5; i++)
      scanf("%d", &arr[i]);
   printf("Array elements are:");
   for(i = 0; i < 5; i++)
   {
      printf("%d ", arr[i]);
      sum+=arr[i];
   }
   printf("\nSum of above elements=%d", sum);
}
```

Output

Enter 5 array elements:9 8 1 5 6
Array elements are:9 8 1 5 6
Sum of above elements=29

Finding the Biggest Element in the Array

```
void main()
{
   int arr[5],big,i;
   printf("Enter 5 array elements:");
   for(i = 0; i < 5; i++)
      scanf("%d", &arr[i]);
   big=arr[0];
   for(i = 1; i < 5; i++)
   {
      if(arr[i]>big)
         big=arr[i];
   }
   printf("Biggest Element of Array is %d", big);
}
```

Output

Enter 5 array elements:8 3 5 7 9
Biggest Element of Array is 9

Reversing the given array

```
void main()
{
   int arr[10],i,j,n,t;
   printf("Enter no. of elements:");
   scanf("%d",&n);
   printf("Enter %d no. of elements:",n);
   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);
```

Search for an Element in an Array

```
void main()
{
   int arr[5],key,i;
   printf("Enter 5 array elements:");
   for(i = 0; i < 5; i++)
      scanf("%d", &arr[i]);
   printf("Enter the search element:");
   scanf("%d",&key);
```

```
    printf("Given Array:");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    for(i = 0, j=n-1; i < n; i++,j--)
    {
        if(i==j || i>j)
            break;
        t=arr[i];
        arr[i]=arr[j];
        arr[j]=t;
    }
   printf("\nReversed Array:");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
}
```

Output

Enter no. of elements:6
Enter 6 no. of elements:2 7 6 8 1 9
Given Array:2 7 6 8 1 9
Reversed Array:9 1 8 6 7 2

```
    for(i = 0; i < 5; i++)
    {
        if(key==arr[i])
        {
            printf("\nElement found at position %d",i+1);
            break;
        }
    }
    if(i==5)
        printf("\nElement not found");
}
```

Output

Enter 5 array elements:2 9 5 7 8
Enter the search element:7
Element found at position 4

Sorting Elements in an Array

```
void main ()
{
    int i, j,temp;
    int arr[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
    for(i = 0; i<10; i++)
    {
        for(j = i+1; j<10; j++)
        {
            if(arr[j] > arr[i])
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    printf("Printing Sorted Array ...\n");
    for(i = 0; i<10; i++)
        printf("%d\t",arr[i]);
}
```

Output

Printing Sorted Array ...
101 78 44 34 23 23 12 10 9 7

Removing Duplicates in an Array

```
void main ()
{
    int i, j, k, n=10;
    int arr[] = { 10, 9, 7, 10, 23, 9, 17, 78, 10, 23};
    for(i = 0; i<n; i++)
    {
        for(j = i+1; j<n; j++)
        {

            if(arr[j] == arr[i])
            {
                for(k=j;k<n-1;k++)
                    arr[k]=arr[k+1];
                n--;
                j--;
            }
        }
    }
    printf("Printing Duplicates removed Array ...\n");
    for(i = 0; i<n; i++)
        printf("%d\t",arr[i]);
```

Output

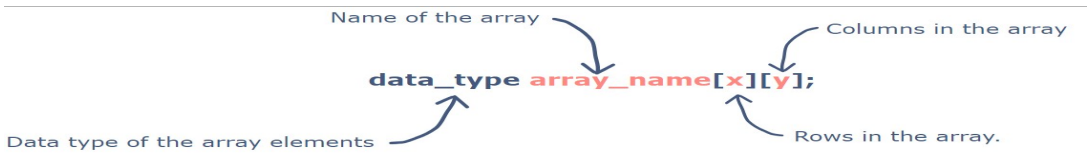
Printing Duplicates removed Array ...
10 9 7 23 17 78

TWO DIMENSIONAL ARRAYS

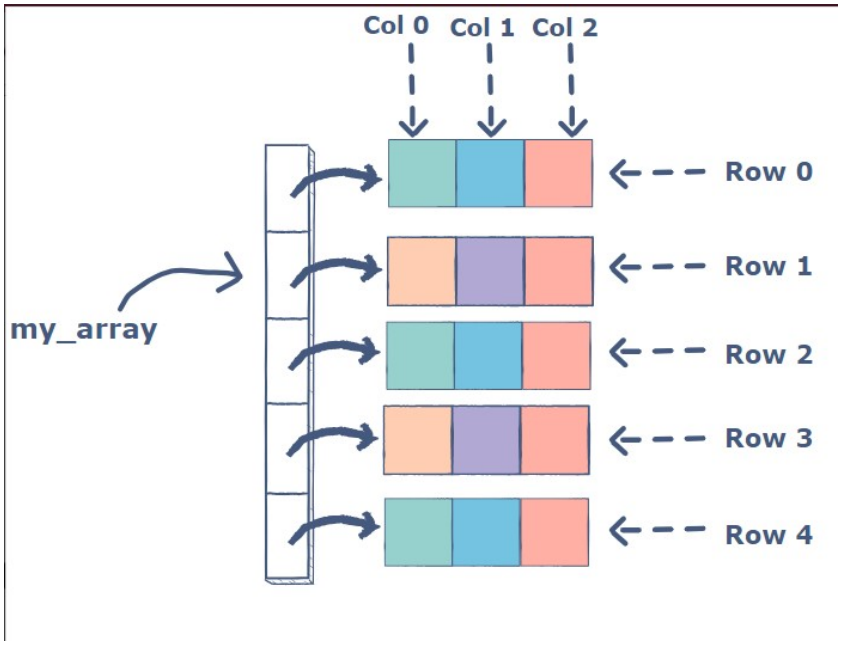
A two-dimensional array in c is one type of array. The 2d array is an array that is organized in rows and columns. The first dimension refers to the row size and second dimension refers to column size. A two-dimensional array is simply, a list of one-dimensional arrays.

Declaration of two-dimensional Array

The syntax to declare the 2D array is given below.



For example, the 2D array with five rows, each row has three columns is : **int my_array[5][3];**
It declares a one-dimensional array that has five elements. Each of these five elements is themselves, an array of three integers. The following diagram shows the memory model of a 2D Array declared:



Initialization of two-dimensional Array
a two-dimensional array in C can be initialize in any one of the following two ways:
Method 1 To initialize a two-dimensional array in C of size x * y, without using any nested brace. The syntax is below. **int Arr[x][y] = {element 1, element 2, ... element xy}**
Example: int A[2][3] = {3, 2, 1, 8, 9, 10}
Method 2 Two-dimensional array in C can also be initialized using nested braces, which makes the visualization of each row and column a bit more easier. The syntax is as follows-

int Arr[x][y] = {{ele 1, ele 2, .. ele y} , {.....} , {..., ele xy-1, ele xy}};

Example: int A[2][3] = {{3, 2, 1}, {8, 9, 10}};

Accessing elements in two-dimensional arrays
In two-dimensional arrays a row and a column index are needed to access a particular element in the array. The general Form is: **array_name[x][y]** Where x: is the row index. y: is the column index.
For example − int val = a[2][3];
The above statement will take the 4th element from the 3rd row of the array.

The following program illustrates the use of 2D Arrays.

```
void main()
{
   int arr[3][3] = {10, 20, 30, 40, 50, 60,70, 80, 90};
   // accessing and printing the elements
   for ( int i = 0; i < 3; i++ )
   {
     // variable i traverses the rows
     for ( int j = 0; j < 3; j++ )
     {
       // variable j traverses the columns
       printf("arr[%d][%d] = %d\n", i,j, arr[i][j] );
     }
   }
}
```

Output

```
arr[0][0] = 10
arr[0][1] = 20
arr[0][2] = 30
arr[1][0] = 40
arr[1][1] = 50
arr[1][2] = 60
arr[2][0] = 70
arr[2][1] = 80
arr[2][2] = 90
```

STRINGS

The string can be defined as the one-dimensional array of characters terminated by a null ('\0'). The character array or the string is used to manipulate text such as words or sentences. Each character in the array occupies one byte of memory, and the last character must always be NULL Character. The termination character ('\0') is important as it identifies where the string ends. When a string as char s[10] is declared, the character s[10] is implicitly initialized with the null in the memory.

STRING DECLARATION

To declare a string in C, the programmer needs to declare a one-dimensional array by specifying the type of the elements of array as character and the number of elements required in array as follows −

char *string_Name[size];*

In the above syntax string_name is any name given to the string variable and size is used to define the length of the string, i.e the number of characters strings will store.

Example: char city[10];

Complier will assign a null character '\0' at the end of the string. So the size should be the maximum of characters to store in the string +1.

STRING INITIALIZATION

A String can be initialized in 4 different ways which are as follows:

1. Assigning a String Literal without Size

String literals can be assigned without size. Here, the name of the string **str** holds the first byte address in array. Ex:- **char str[] = "Narayana";**

2. Assigning a String Literal with a Predefined Size

String literals can be assigned with a predefined size. But it is required to add one extra space for null character. Ex: - **char str[50] = "Narayana";**

3. Assigning Character by Character with Size

A String can be assigned character by character by specifying explicitly end character as '\0' which is a null character. Ex:- **char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};**

4. Assigning Character by Character without size

A String can be assigned character by character without size with the NULL character at the end. The size of the string is determined by the compiler automatically. Ex:- **char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};**

STRING INPUT

C language allows reading the string as input from the user. The two important functions that support string input are : scanf() and gets().

The **scanf( )** function with `%s' format specifier can be used to read a string from the standard input. The general form is – **scanf("%s", Array_Name);**

Example program using scanf( )

```
void main()
{
   char clr[20];
   printf("Enter your favorite color: ");
   scanf("%s",clr)
   printf("Your favorite color is: %s", clr);
}
```

Output

Enter your favorite color: Orange
Your favorite color is: Orange

The problem with scanf( ) is, it doesn't accept white space characters.

The C standard library function, **gets()** can be used to take a complete line as input and stores it in the string provided as a parameter. The working of gets() is such that it keeps reading the input stream until it encounters a newline character(\n).

General form - **gets(Array_Name)**

Example program using gets( )

```
void main()
{
   char ln[30];
   printf("Enter a Sentence: ");
   gets(ln);
   printf("Your Sentence is: %s", ln);
}
```

Output

Enter a Sentence: This is India
Your Sentence is: This is India

PRINTING STRINGS

C language allows writing the strings as output to the user. The two important functions that support string printing are : printf() and puts().

The **printf( )** function with `%s' format specifier can be used to write a string to the standard output. The general form is – **printf("%s", Array_Name);**

Example program using printf( )

```
void main()
{
   char clr[20];
   printf("Enter your favorite color: ");
   scanf("%s",clr)
   printf("Your favorite color is: %s", clr);
}
```

Output

Enter your favorite color: Orange
Your favorite color is: Orange

The C standard library function**, puts()** can be used to print a line of text as ouput which is provided as parameter.

General form - **puts(Array_Name)**

Example program using puts( )

```
void main()
{
   char ln[30];
   printf("Enter a Sentence: ");
   gets(ln);
   printf("Your Sentence is: ");
   puts(ln);
}
```

Output

Enter a Sentence: This is India
Your Sentence is: This is India

CHARACTER FUNCTIONS IN C

C has functions to test or manipulate individual characters. The functions are all defined in the ctype.h header file. To use the CTYPE functions, the ctype.h header file must be included in the source code:

#include <ctype.h>

CTYPE functions are of two categories: Testing and Manipulation.

The following table lists some of the Testing Functions.

Function	Returns TRUE When 'ch' is
isalnum(ch)	A letter of the alphabet (upper- or lowercase) or a number
isalpha(ch)	An upper- or lowercase letter of the alphabet
isascii(ch)	An ASCII value in the range of 0 through 127
isblank(ch)	A tab or space or another blank character
isdigit(ch)	A character 0 through 9
islower(ch)	A lowercase letter of the alphabet, a to z
isupper(ch)	An uppercase letter of the alphabet, A to Z

The following table lists manipulation functions.

Function	Description
toascii(ch)	Gives the ASCII code value of ch, in the range of 0 through 127
tolower(ch)	Gives the lowercase of character ch
toupper(ch)	Gives the uppercase of character ch

All the above functions accept an integer as argument and returns and integer as output.

STRING MANIPULATION FUNCTIONS

C supports several pre-defined string handling functions, that can be used to make string manipulations. The following are the most used string handling functions.

Function Name	Description	General Form
String Length Function	returns the length of the given string. It doesn't count null character '\0'.	strlen(string_name)
String Copy Function	copies the contents of source string to destination string.	strcpy(destination, source)

Function Name	Description	General Form
String Concatenation Function	joins first string with second string. The result of the string is stored in first string.	strcat(first_string, second_string)
String Comparison Function	compares the first string with second string. If both strings are same, it returns 0.	strcmp(first_string, second_string)
String Reversing Function	returns reverse string	strrev(string)
String Lower Conversion Function	returns string characters in lowercase	strlwr(string)
String Higher Conversion Function	returns string characters in uppercase	strupr(string)

Example Program using String Handling Functions

```
void main()
{
   char s1[]="Hello", s2[]="NECG",t[10];
   printf("Length of %s is %d\n",s1,strlen(s1));
   strcpy(t,s2);
   printf("Copied String t=%s\n",t);
   strcat(s1,s2);
   printf("Joined String s1=%s\n",s1);
   printf("Comparison of t and s2 is %d (EQUAL)\n",strcmp(t,s2));
   //printf("Reverse of NECG is %s",strrev(s2));
   //printf("NECG in Lower is %s",strlwr(t));
}
```

Output

```
Length of Hello is 5
Copied String t=NECG
Joined String s1=HelloNECG
Comparison of t and s2 is 0 (EQUAL)
Reverse of NECG is GCEN
NECG in Lower is necg
```

ARRAY OF STRINGS

An array of strings in C is a two-dimensional array of character types. Each String is terminated with a null character (\0). It is an application of a 2d array.

The Syntax of **declaring an array of strings** is shown below:

char variable_name[R][C];

Here, **var_name** is the name of the variable in C.
R is the maximum number of strings that can be stored in the array of strings.
C is the maximum number of characters that can be stored in a string.

Example: sports[5][15];

The above declaration creates an array holds 5 strings where each string is a length of 15 characters.

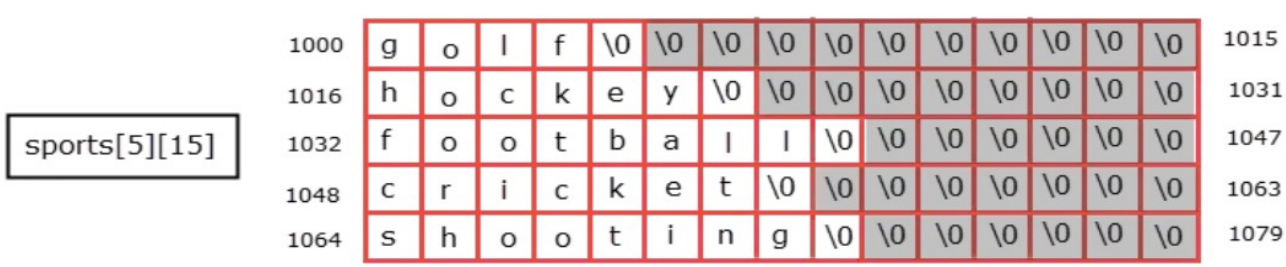
The Syntax of **initializing an array of strings** is shown below:

char variable_name[R][C] = {Str1, Str2,……., StrN};

Here, the no. of strings is equal to R i.e, and is optional.

Example: sports[5][15]={ “golf”, “hockey”, “football”, “cricket”, “shooting” };

The memory representation of array of strings for above example is shown below.



The Disadvantage of using 2D Arrays for array of strings is the memory allocated to all the Strings will be the size of the longest String, causing "Memory wastage". The darkened part in the above representation is the memory wasted.

Example program using 2D Arrays for Array of Strings

```
void main()
{
   int i;
   char str[3][10] = {"Hello", "Black", "Apple"};
   printf("Strings Array: \n");
   for(i = 0; i < 3; i++)
   {
      printf("%s\n", str[i]);
   }
}
```

Output

```
String Array:
Hello
Black
Apple
```